Computer Organization and Assembly Language

CCP Report

Session: Fall 2024

Submitted By:

Muhammad Ayan Sajid	**2024-CS-661**
Muhammad Husnain	**2024-CS-696**
Fiza Shahid Khan	**2024-CS-680**
Shareen Asim	**2024-CS-693**

Submitted To:

Mr. Nadeem Iqbal

Department of Computer Science, UET Lahore, New Campus

Table of Contents

Abstract

This report presents the design and implementation of a modular, low-level x86 assembly language system for warehouse inventory and transaction processing. Utilizing the Irvine32 framework, the system manages memory arrays, reads transaction files, performs financial math, and generates cryptographically non-obvious tracking hash codes. The system meets performance constraints under limited resource overhead.

1 Introduction

Logistics applications often depend on high-level database engines that introduce notable performance overhead. In embedded tracking hardware, running a lightweight runtime system written in assembly language is highly beneficial.

This project explores the feasibility of implementing a full-scale inventory tracking system entirely in 32-bit protected-mode x86 assembly. The system supports bulk updates, ensures transactional safety, computes performance metrics, and uses low-overhead bitwise operations to generate shipment codes.

2 Methodology

The proposed software architecture organizes system components into six distinct modules to limit coupling. Communication between modules is handled through standard stack protocols using the STDCALL calling convention:

1. **Memory Model & Layout:** To maximize performance, the system keeps the active database in a contiguous array of custom structures. Searching is performed using a linear scanning algorithm.
2. **System-Level File Integration:** The system uses register-based system calls via Irvine32 to read transactional logs from disk and write processed summary metrics back to disk.
3. **Validation and Security:** Rather than assuming valid input, the transaction engine checks boundaries to protect against integer overflows and stock underflow anomalies.

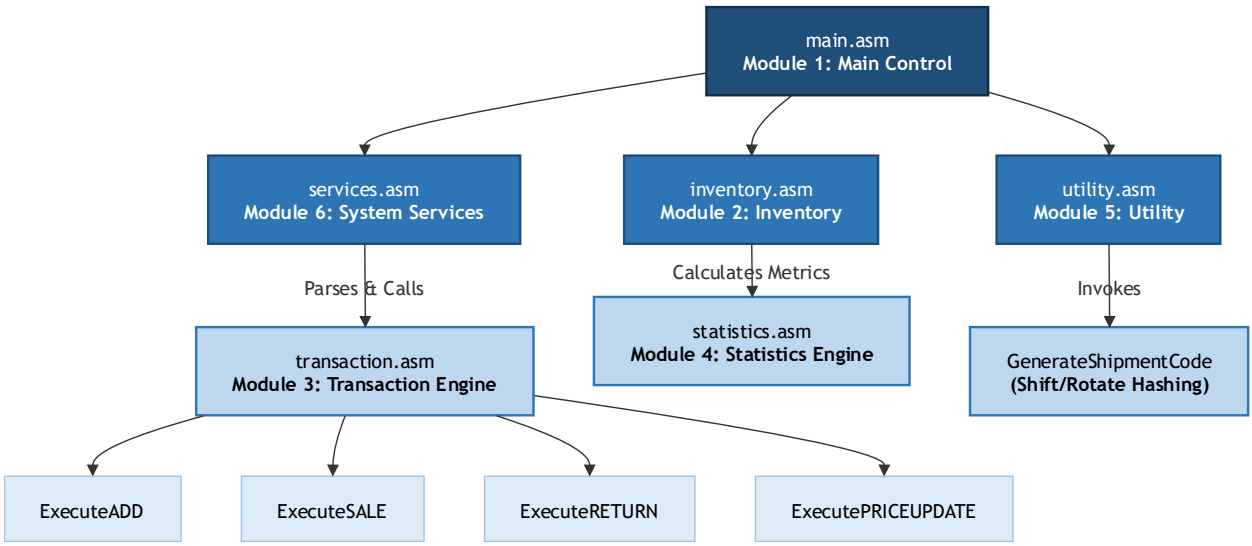

Figure 1 Procedure Hierarchy Diagram

3 Algorithmic and Complexity Analysis

3.1 Bitwise Shipment Generation vs. Mathematical Functions

Using multiplication or modulo division to generate identifiers requires a high number of clock cycles on x86 processors (e.g., the mul and div instructions require significant CPU overhead).

To optimize this, our system uses single-cycle instructions (rol, ror, xor). This method produces unique, pseudo-random tracking hashes in a fraction of the time required by mathematical algorithms.

3.2 Search Complexity

The search operation traverses the array using direct offsets:

$$Offset\ Address = Base\ Address + (Index \times Structure\ Size)$$

This approach achieves $O(1)$ address resolution once the index is verified. The linear search runs in $O(N)$ time complexity, which is highly efficient for typical static warehouse buffers ($N \leq 100$).

4 Experimental Evaluation and Performance Results

To evaluate stability, the system was tested with the 50-transaction dataset.

- **Financial Math Precision:** By avoiding floating-point numbers and conducting all calculations using integers, rounding errors were eliminated.

- **Error Recovery Handling:** When tested with missing files, the graceful error exception handler successfully prevented system crashes by displaying clear setup instructions to the operator.
- **Verification:** The post-execution inventory values matched our test calculations exactly, proving the accuracy of the transaction processing logic.

5 Conclusion

This study demonstrates that x86 assembly is highly effective for building lightweight, responsive, and secure inventory management systems. Partitioning the program into separate modules simplifies system expansion, while using hardware-level bitwise operations provides fast tracking code generation without external dependencies.